

二、线程与线程池封装

1、基础

之前在Linux阶段，我们开始学过进程的相关概念、后面又学了线程的相关概念，现在我们主要的来进行线程的进一步学习。之前在Linux里面，使用的是C语言面向过程的思想，从现在开始我们需要使用C++面向对象的思想进行封装，但是在封装之前我们先来回顾一下线程相关的API。

1.1、线程的创建

```
1 #include <pthread.h>
2
3 int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
4                   void *(*start_routine) (void *), void *arg);
5 thread: 线程id
6 attr: 线程属性，默认为空
7 start_routine: 线程入口函数
8 arg: 线程入口函数的参数，默认可以使用空
```

1.2、线程终止

```
1 #include <pthread.h>
2
3 功能: 线程终止(注意: 进程终止是exit函数)
4 void pthread_exit(void *value_ptr);
5 参数value_ptr:value_ptr不要指向一个局部变量。
6 返回值: 无返回值, 跟进程一样, 线程结束的时候无法返回到它的调用者(自身)
```

1.3、线程等待

```
1 #include <pthread.h>
2
3 int pthread_join(pthread_t thread, void **value_ptr);
4 参数thread:线程ID
5 value_ptr:它指向一个指针, 后者指向线程的返回值
6 返回值: 成功返回0; 失败返回错误码
```

为什么需要线程等待?

- 已经退出的线程，其空间没有被释放，仍然在进程的地址空间内。
- 创建新的线程不会复用刚才退出线程的地址空间。
- 主线程需要知道所创建的新线程是否有完成任务。并且可以避免像僵尸进程的问题

1.4、线程取消

```
1 #include <pthread.h>
2
3 int pthread_cancel(pthread_t thread);
4 参数thread:线程ID
5 返回值: 成功返回0; 失败返回错误码
```

调用该函数的线程将挂起等待,直到id为thread的线程终止。thread线程以不同的方法终止,通过pthread_join得到的终止状态是不同的,总结如下:

- 如果thread线程通过return返回, value_ptr所指向的单元里存放的是thread线程函数的返回值。
- 如果thread线程被别的线程调用pthread_cancel异常终掉,value_ptr所指向的单元里存放的是常数PTHREAD_CANCELED (这是一个宏定义, 本质就是-1)。
- 如果thread线程是自己调用pthread_exit终止的,value_ptr所指向的单元存放的是传给pthread_exit的参数
- 如果对thread线程的终止状态不感兴趣,可以传NULL给value_ptr参数

1.5、线程分离

```
1 | int pthread_detach(pthread_t thread);
```

- 默认情况下, 新创建的线程是joinable的, 线程退出后, 需要对其进行pthread_join操作, **否则无法释放资源, 从而造成系统泄漏。**
- 如果不关心线程的返回值, join是一种负担, 这个时候, 我们可以告诉系统, 当线程退出时, 自动释放线程资源。

可以是线程组内其他线程对目标线程进行分离, 也可以是线程自己分离:

```
pthread_detach(pthread_self());
```

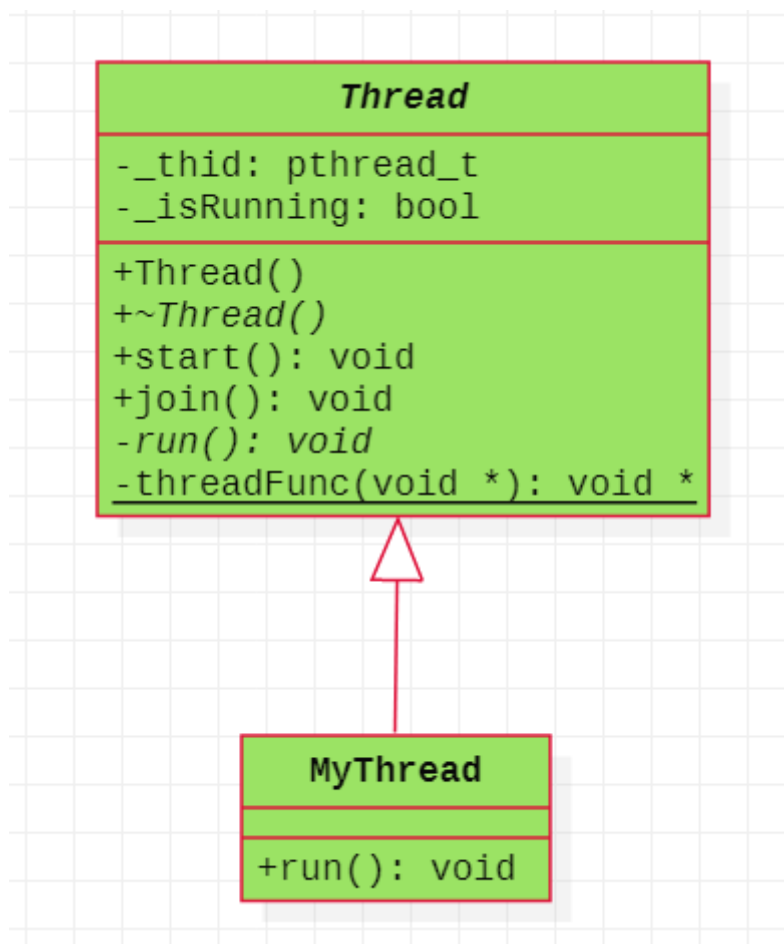
线程分离以后, 主线程以后就再也不需要关心这个线程了, 在这个线程结束以后, 会自动的进行资源释放。但是这里有一点: **如果这个分离的线程出现了问题, 依旧会导致进程的退出。**

2、面向对象与基于对象

前面我们说过, 面向对象具有四大基本特征: 抽象、封装、继承、多态, 所以我们可以从面向对象的思想对线程进行封装。

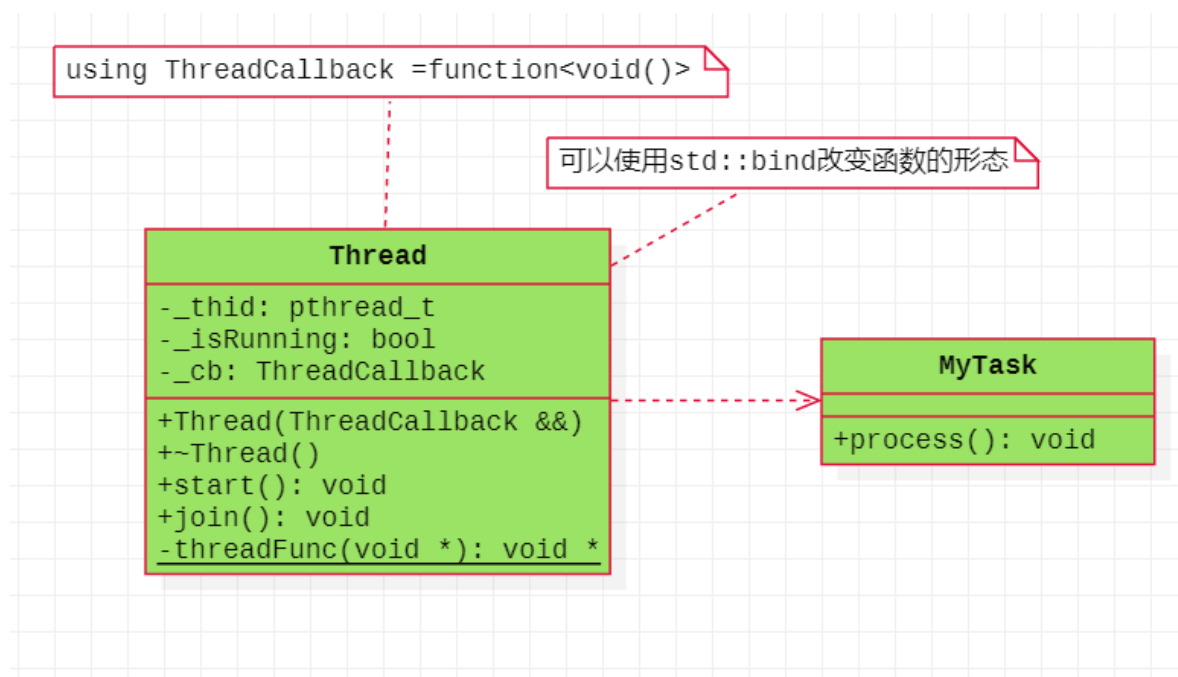
2.1、面向对象的线程封装

有了抽象的概念之后, 现在我们将线程抽象成一个类, 用类的观点来看线程。然后用线程类去创建具体的对象, 线程对象与线程对象之间进行交互, 通信。接下来就为线程类设置成员, 每个线程会有线程号, 可以设置为数据成员; 线程是否开始运行、线程是否结束, 对应会有start函数与join函数; 线程入口函数可以封装为一个函数threadFunc, 然后具体需要执行的任务可以封装为一个run方法, 具体的业务留给Thread的派生类去实现, 所以run方法可以设置为虚方法, Thread类也就成为抽象类了。为了实现具体的任务方法, 可以在用一个类去继承抽象类Thread, 并实现虚方法run, 所以可以设置为下面的关系图。



2.2、基于对象的线程封装

解决问题的思维方式很多，除了之前C语言的面向过程的方法，以及上面的面向对象的方法之外，换一种方式。我们前面学习过，类和类之家的关系除了继承，还有一些其他的关系，所以可以食欲组合与依赖的方式，也就是基于对象的方法（不用使用继承）。因为没有继承，所以需要把Thread类中的抽象方法run()函数通过function与bind的方式进行修改即可。之前的run方法是一个返回类型是void参数为空的函数，现在直接使用bind改变函数的形态，将Thread变为一个非抽象类。然后将任务通过参数传给Thread的构造函数，当然这个参数可以使用对应的数据成员进行接收，这个任务可以交给其他的类来完成，并且打包给Thread线程进行执行，设计如下：



三、生产者与消费者问题

生产者与消费者问题，其实也就是多个线程的问题，但是会涉及到线程之间对共享资源的互斥访问，所以需要互斥锁与条件变量的准备知识，接下来我们先看看互斥锁与条件变量。

1、互斥锁mutex

1.1、数据混乱的三个原因

- 资源共享（独享资源则不会出现混乱）
- 调度随机（意味着数据访问会出现竞争）
- 线程间缺乏必要的同步机制

总结：前面两个条件是不能改变的。

1.2、互斥锁的创建

```
1 pthread_mutex_t mutex; //直接定义一把互斥锁
```

1.3、互斥锁的初始化

```
1 #include <pthread.h>
2 int pthread_mutex_init(pthread_mutex_t *restrict mutex,
3                         const pthread_mutexattr_t *restrict attr);
4 功能： 初始化互斥锁
5 mutex: pthread_mutex_t类型的变量的地址，也就是互斥锁的名字。
6 mutexattr: pthread_mutexattr_t类型的变量地址，用于设置锁的属性，默认可以设为NULL；
7
8 返回： 成功0，错误返回错误代码。
9 静态初始化: pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
10 动态初始化: pthread_mutex_init(&mutex, nullptr);
```

1.4、互斥锁的销毁

```
1 int pthread_mutex_destroy(pthread_mutex_t *mutex);
2 功能： 销毁互斥锁
3 参数： mutex为要销毁的互斥锁的地址
4 返回： 成功0，错误返回错误代码
```

1.5、互斥锁的加锁

```
1 //这个操作是阻塞调用的，如果这个锁此时正在被其它线程占用，那么函数调用会进入到这个锁的排队队列中，并会进入阻塞状态，直到拿到锁之后才会返回
2 int pthread_mutex_lock(pthread_mutex_t *mutex);
3
4 //会尝试对互斥量加锁，如果该互斥量已经被锁住，函数调用失败，返回EBUSY，否则加锁成功返回0，线程不会被阻塞
5 int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

1.6、互斥锁的解锁

```
1 int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

2、条件变量condition

2.1、条件变量的创建

```
1 pthread_cond_t cond; //定义一个条件变量
```

2.2、条件变量的初始化

```
1 #include <pthread.h>
2 int pthread_cond_init(pthread_cond_t *restrict cond,
3                        const pthread_condattr_t *restrict attr);
4 功能: 初始化条件变量
5 cond: 条件变量名的地址
6 cond_attr: 条件变量的属性
7
8 返回: 成功0, 错误返回错误编号
9
10 静态初始化: pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
11 动态初始化: pthread_cond_init(&cond, nullptr);
```

2.3、条件变量的销毁

```
1 #include <pthread.h>
2 int pthread_cond_destroy(pthread_cond_t *cond);
3 功能: 销毁条件变量
4 cond: 条件变量名的地址
5
6 返回: 成功0, 错误返回错误代码
```

2.4、条件变量的等待

```
1 #include <pthread.h>
2 int pthread_cond_wait(pthread_cond_t *restrict cond,
3                       pthread_mutex_t *restrict mutex);
4 功能: 等待互斥锁mutex下的条件cond的发生
5 cond: 要等待的条件变量
6 mutex: 保护条件变量的互斥锁
7
8 返回: 成功0, 错误返回错误代码
9
10 //函数作用:
11 //1、阻塞等待条件变量cond满足。
12 //2、释放已掌握的互斥锁(解锁互斥量) 相当于pthread_mutex_unlock(&mutex); 第1,2步为一个原子操作
13 //3、当被唤醒, pthread_cond_wait函数返回, 解除阻塞并重新获得互斥锁
14 pthread_mutex_lock(&mutex)
15
16 int pthread_cond_timedwait(pthread_cond_t *restrict cond,
17                             pthread_mutex_t *restrict mutex,
18                             const struct timespec *restrict abstime);
```

2.5、条件变量的唤醒

```
1 int pthread_cond_signal(pthread_cond_t *cond);
```

```
2  功能：发送条件变量cond
3  参数：cond为要发送的条件变量
4
5  返回：成功0，否则返回错误编号
6  备注：唤醒至少一个阻塞在条件变量上的线程
7
8  int pthread_cond_broadcast(pthread_cond_t *cond);
9  功能：发送条件变量cond
10 参数：cond为要发送的条件变量
11
12 返回：成功0，否则返回错误编号
13
14 备注： 通知所有等待该条件的线程
```

3、生产者消费者问题

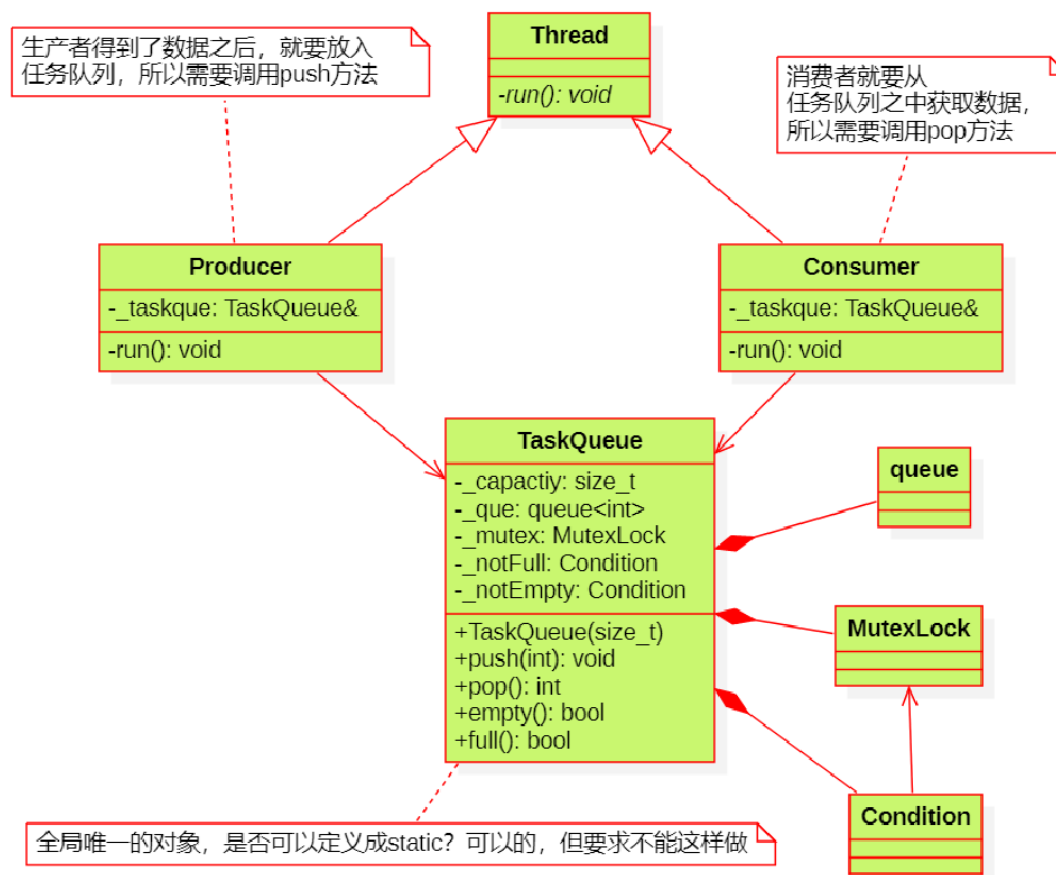
3.1、禁止复制

在C++的前面，我们讲过两种语义，值语义与对象语义，特别是对于对象语义，表示的是不能进行复制或赋值，所以我们可以将类的拷贝构造函数与赋值运算符函数删除或者设置为私有，但是除了这种方式之外，为了表示禁止复制的概念，我们还可以使用什么方法呢？因为具有对象语义的有很多，不可能每个类中都把拷贝构造函数与赋值运算符函数都设置为私有的，这样比较麻烦，所以可以想象其他的办法，使用继承。

```
1  class NoCopyable
2  {
3  protected:
4      NoCopyable() { }
5      ~NoCopyable() { }
6      NoCopyable(const NoCopyable &) = delete;
7      NoCopyable &operator=(const NoCopyable &) = delete;
8  };
9
10 class Derived
11 : public NoCopyable
12 {
13
14 };
15
16 void test()
17 {
18     Derived d1;
19     Derived d2(d1); //error
20     Derived d3;
21     d3 = d1; //error
22 }
```

3.2、面向对象的生产者消费者封装

生产者与消费者问题，是一个经典的常规问题，其实也是线程问题。可以把生产者看成是一个线程，也可以把消费者看成是一个线程，两个线程都可以继承自之前上面封装的抽象类Thread，然后再实现虚方法run。当然，因为生产者与消费者需要从共享的仓库中存数据或者取数据，设计到对仓库的互斥访问，所以需要加锁，当然在这里我们把仓库用一个任务队列TaskQueue进行封装，提供互斥锁、条件变量的基本数据成员，当然任务队列里面也就是基本操作，队列是不是满的，是不是空的，存数据与取数据等基本操作。当然，之前的互斥锁与条件变量是可以进行封装的，全部使用面向对象的思想，抽象为类，于是就可以画出如下类图。



利用以上的类图就可以实现生产者与消费者的代码实现了。

3.3、基于对象的生产者与消费者封装

可以将面向对象的生产者消费者改为基于对象的，也就是使用function + bind的方法。

四、线程池

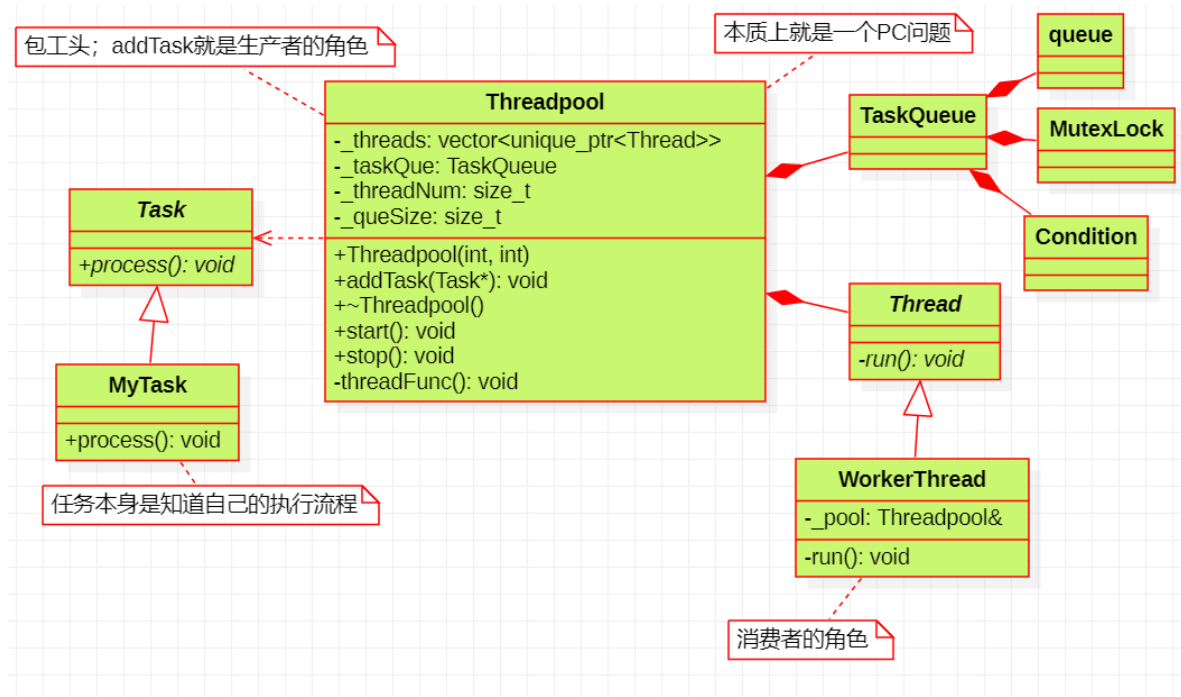
4.1、线程池基础概述

为什么要有线程池？假设没有使用线程池时，一个请求用一个子线程来处理。每来一个请求，都得创建子线程，子线程执行请求，关闭子线程。当请求量（并发）比较大的时候，频繁地创建和关闭子线程，也是有开销的。因此提出线程池，提前开辟好N个子线程，当有任务过来的时候，先放到任务队列中，之后N个子线程从任务队列中获取任务，并执行，这样能大大提高程序的执行效率。其实当任务数大于线程池中子线程的数目的时候，就需要将任务放到缓冲区（队列）里面，所以本质上还是一个生产者消费者模型。

查看线程的状态的命令

```
1 | ps -eLf | grep xxx
```

4.2、面向对象的线程池封装



在代码实现的过程中，会导致线程池无法退出，大家可以分析原因？这其中会涉及一个概念，称为虚假唤醒。那什么是虚假唤醒呢？可以看看下面解释。

虚假唤醒：POSIX规范为了简化pthread_cond_signal的实现，允许它在实现的时候唤醒一个以上的线程。（《UNIX环境高级编程》第三版）。结果是，当一个线程调用pthread_cond_signal后，多个调用pthread_cond_wait或pthread_cond_timedwait的线程返回。

懂得了什么是虚假唤醒后，就能解决线程池无法退出的问题了？大家看看使用什么样的方式

4.3、基于对象的线程池封装

可以将function与bind结合使用，将虚方法进行改造，也就是不使用继承。

